First-Class Functions

A **first-class function** means that functions in JavaScript are treated as "first-class citizens." This means functions can be:

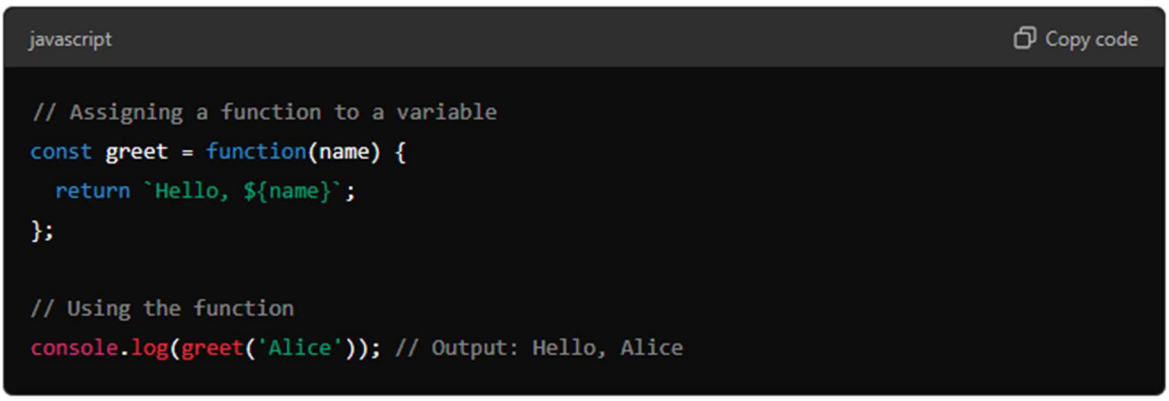
- Assigned to variables

- Passed as arguments to other functions

- Returned from other functions

- Stored in data structures like arrays or objects

Example:




```
// Assigning a function to a variable
const greet = function(name) {
  return `Hello, ${name}`;
};

// Using the function
console.log(greet('Alice')); // Output: Hello, Alice
```

In the example above, the `greet` function is treated like any other variable. It can be passed around and used flexibly, showing that functions in JavaScript have first-class status.

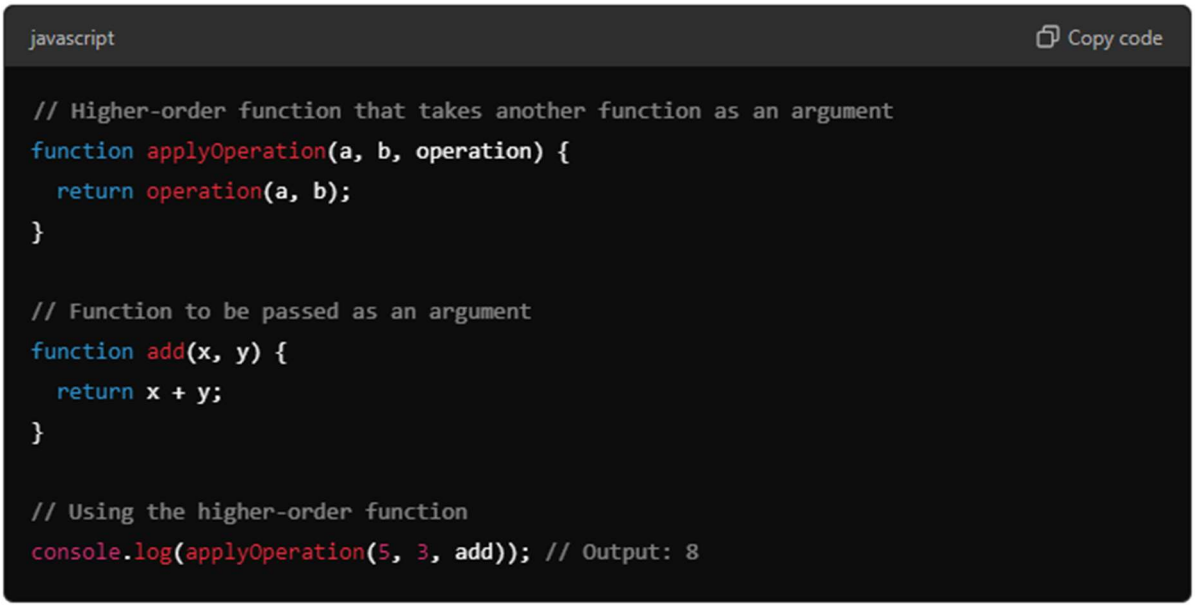
Higher-Order Functions

A **higher-order function** is a function that either:

- Takes one or more functions as arguments, or

- Returns a function as its result

This leverages the first-class nature of functions in JavaScript.




```
// Higher-order function that takes another function as an
argument
function applyOperation(a, b, operation) {
  return operation(a, b);
}

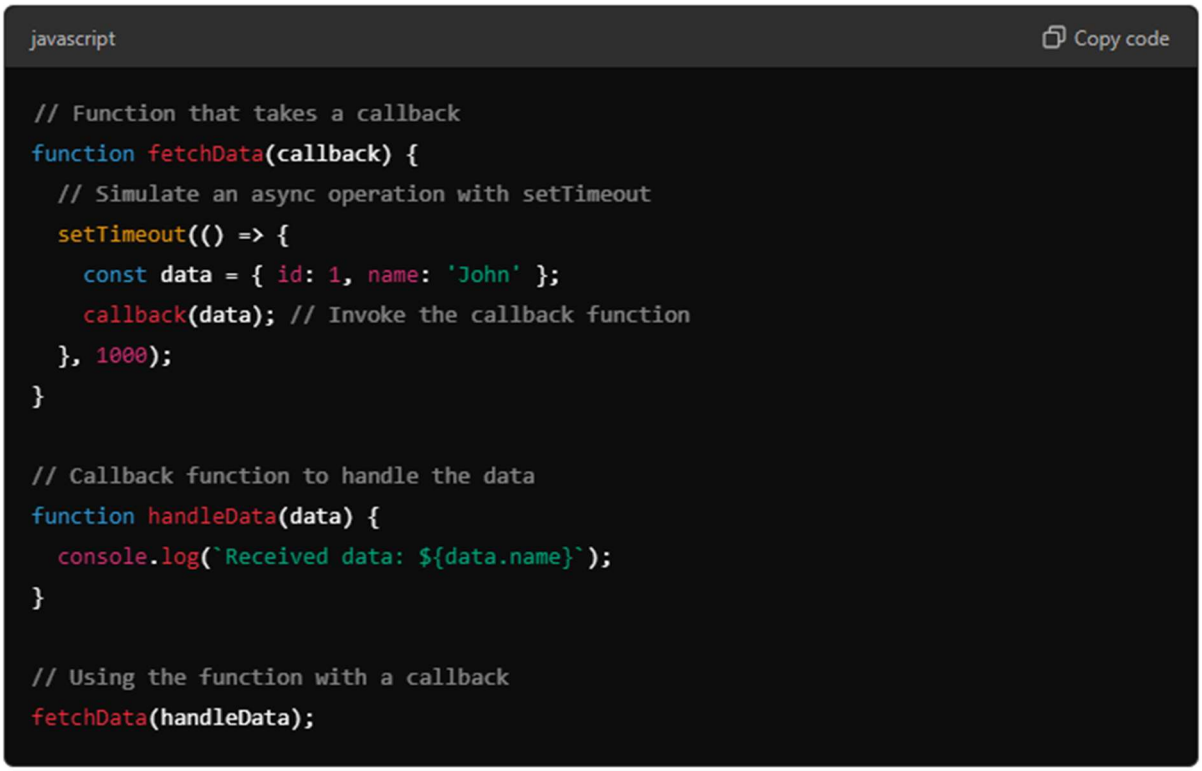
// Function to be passed as an argument
function add(x, y) {
  return x + y;
}

// Using the higher-order function
console.log(applyOperation(5, 3, add)); // Output: 8
```

Here, `applyOperation` is a higher-order function because it takes the `add` function as an argument and uses it to perform an operation.

Callback Functions

A **callback function** is a function that is passed as an argument to another function and is executed (or "called back") at a later time. Callbacks are often used in asynchronous programming or to handle events.




```
// Function that takes a callback
function fetchData(callback) {
  // Simulate an async operation with setTimeout
  setTimeout(() => {
    const data = { id: 1, name: 'John' };
    callback(data); // Invoke the callback function
  }, 1000);
}

// Callback function to handle the data
function handleData(data) {
  console.log(`Received data: ${data.name}`);
}

// Using the function with a callback
fetchData(handleData);
```

In this example, `fetchData` takes a callback function (`handleData`), which is executed once the data is fetched asynchronously.

Summary:

- **First-Class Functions**: Functions can be treated like any other variable (assigned, passed around, etc.).

- **Higher-Order Functions**: Functions that take other functions as arguments or return functions.

- **Callback Functions**: Functions passed as arguments and executed later (often in response to an event or after an asynchronous operation).

Callbacks are a specific use case of higher-order functions.